🎤 SPEECH DE DEFENSA ORAL — UNIDAD 3: ADMINISTRACIÓN DE MEMORIA

> **Sistemas Operativos (UADE / Red Hat)** — Guion hablado paso a paso con analogías sencillas para exponer en clase (2 a 3 minutos).

⚡ 1. Resumen Relámpago (Si tenés 15 segundos para resumir tu tema)

- **El Problema:** En nuestro servidor Red Hat queríamos cargar un proceso de **426 KB** (`P4`), teníamos **959 KB libres en total** (más del doble), pero el sistema lo **rechazaba por falta de memoria**.
- **La Causa Raíz:** La memoria usaba **asignación contigua con First-Fit**, y esos 959 KB estaban partidos en 5 huecos sueltos donde ninguno llegaba a 426 KB seguidos (**Fragmentación Externa**).
- **La Solución:** En C comparamos **First-Fit** contra **Best-Fit** (que acomoda cada proceso en el hueco más justo y logra meter los 4 procesos), y explicamos por qué Linux moderno lo resuelve de raíz con **Paginación y Memoria Virtual**.

🎮 2. La Analogía Cotidiana / Gamer (Para romper el hielo)

> 🗣️ **Cómo contarlo fácil:** *"Para entender lo que pasa en esta unidad, imagínense el **inventario por casilleros del Resident Evil 4 o del Diablo** (o estacionar autos en la calle). Supongamos que en tu mochila tenés **10 cuadraditos libres en total**, pero están todos separados: 2 arriba, 3 al medio, 2 abajo y 3 al costado. Si querés guardar un ítem grande que ocupa **4 cuadraditos seguidos**, el juego te dice **'No hay espacio'**, aunque te sobre lugar en total. Eso en Sistemas Operativos se llama **Fragmentación Externa** en asignación contigua."*

🎙️ 3. El Speech Completo Paso a Paso (~2 Minutos)

Parte 1: El Problema en el Servidor (30 seg)

> 🗣️ **Decís:** *"En la Unidad 3 trabajamos sobre Administración de Memoria. Planteamos el caso de un servidor en Red Hat donde corrían varios procesos y quedaban **5 huecos libres en la memoria RAM**: uno de 100 KB, uno de 500, uno de 200, uno de 300 y uno de 600 KB (en total **1.700 KB libres**). Cuando llegaban 4 procesos nuevos pidiendo memoria contigua, el último proceso (`P4`, de **426 KB**) tiraba error de memoria insuficiente, a pesar de que cuando mirábamos la memoria libre sobraban **959 KB**."*

Parte 2: Mostrando el Fallo en la Consola de Red Hat — First-Fit (30 seg)

`[👉 EN CONSOLA: Ejecutar ./ejecutar_tpo.sh y señalar el bloque "1. EJECUCION CON PRIMER AJUSTE (FIRST-FIT)"]`

> 🗣️ **Decís:** *"Acá cuando corremos `./ejecutar_tpo.sh`, primero se ejecuta nuestro simulador en C con el algoritmo **First-Fit (Primer Ajuste)**. ¿Qué hace First-Fit? Mete a cada proceso en el **primer hueco** donde entra:
>
> - Al proceso 1 (212 KB) lo mete en el Hueco 2 (que era de 500 KB), partiéndolo al medio.
> - Al proceso 2 (417 KB) lo mete en el Hueco 5 (de 600 KB).
> - Al proceso 3 (112 KB) lo mete también en lo que sobraba del Hueco 2. ¿Qué pasa cuando llega `P4` pidiendo **426 KB**? Como ven acá en pantalla, dice **RECHAZADO**. Nos quedaron **959 KB libres en total**, pero repartidos en 5 pedacitos (el más grande es de 300 KB). Como la asignación es **contigua**, no podemos partir el proceso y falla por **Fragmentación Externa**."*

Parte 3: Cómo lo Resuelve nuestro Código en C — Best-Fit (40 seg)

`[👉 EN CONSOLA: Señalar el bloque "2. EJECUCION CON MEJOR AJUSTE (BEST-FIT)"]`

> 🗣️ **Decís:** *"Abajo nuestro script corre el mismo escenario pero cambiando el algoritmo a **Best-Fit (Mejor Ajuste)** (`./simulador_memoria B`). En vez de agarrar el primer hueco que encuentra, Best-Fit revisa los 5 huecos y elige **el hueco donde el proceso entra más justito**, para no desperdiciar los huecos grandes:
>
> - Entonces mete a `P1` (212 KB) en el hueco de 300 KB, y así **guarda intacto el hueco de 600 KB**.
> - Cuando finalmente llega `P4` (426 KB), el hueco de 600 KB sigue libre y **los 4 procesos se asignan con éxito**."*

Parte 4: Medición con `time` y Cierre Teórico de Linux (20 seg)

`[👉 EN CONSOLA: Señalar el bloque "3. ANALISIS COMPARATIVO DE TIEMPOS DE EJECUCION"]`

> 🗣️ **Decís:** *"Por último, en el script medimos el tiempo con `time`. Aunque Best-Fit logró meter los 4 procesos, en la teoría tiene dos desventajas: es más lento porque tiene que recorrer toda la lista de huecos, y suele dejar huequitos minúsculos inservibles. Por eso **Red Hat Linux moderno no usa asignación contigua para los procesos**, sino **Paginación y Memoria Virtual**: divide la RAM en **marcos de 4 KB** y los procesos en **páginas de 4 KB**, y con la **MMU** y la **Tabla de Páginas** los guarda en cualquier hueco libre sin necesitar que estén seguidos, eliminando al 100% la fragmentación externa."*

💻 4. "Machete" Rápido del Código (Por si el profe te pide abrir `simulador_memoria.c`)

1. **Si te pregunta cómo representamos la memoria y los procesos (líneas 20–22):**
   - *Qué responder: "Usamos dos arreglos de enteros en C: `huecos[5] = {100, 500, 200, 300, 600}` para los bloques libres de RAM y `procesos[4] = {212, 417, 112, 426}` para lo que pide cada proceso."*
2. **Si te pregunta cuál es la diferencia en el código entre First-Fit y Best-Fit (líneas 59–84):**
   - *Qué responder: "En **First-Fit** (línea 65) apenas encontramos un hueco donde `huecos[j] >= procesos[i]`, hacemos un `break` y cortamos la búsqueda. En cambio, en **Best-Fit** (líneas 71–78) no hay `break`: recorremos los 5 huecos comparando `huecos[j] < huecos[mejor_idx]` para quedarnos con el más chico donde quepa."*
3. **Si te pregunta por qué en el `.sh` hacemos `time ./simulador_memoria F > /dev/null`:**

- *Qué responder: "Mandamos la salida de texto a `/dev/null` para medir con `time` únicamente lo que tarda el algoritmo en la CPU, sin que nos ensucie la medición el tiempo de imprimir letras en la pantalla."*

🛟 5. Salvavidas: Las 5 Preguntas Rápidas del Profe en Vivo

❓ 1. "¿Qué diferencia hay entre Fragmentación Externa y Fragmentación Interna?"

> 🗣️ **Respuesta corta:** *"La **Fragmentación Externa** es el espacio que queda **afuera** de las particiones, repartido en huecos sueltos entre procesos, por lo que no podés meter un proceso grande aunque la suma te alcance (pasa en asignación contigua y segmentación). La **Fragmentación Interna** es el pedacito que sobra **adentro** de un bloque o página de tamaño fijo: por ejemplo, si las páginas son de 4 KB y tu proceso usa 3 KB, ese 1 KB que sobra adentro de la página asignada es fragmentación interna."*

❓ 2. "Si Best-Fit pudo meter los 4 procesos, ¿significa que Best-Fit siempre es mejor que First-Fit?"

> 🗣️ **Respuesta corta:** *"No. En este ejemplo puntual ganó porque salvó el hueco de 600 KB, pero en general **First-Fit suele ser más rápido** (porque corta en el primer hueco que sirve) y **Best-Fit tiende a llenar la memoria de micro-huecos diminutos** e inútiles porque siempre busca dejar el menor sobrante posible."*

❓ 3. "¿Cómo se soluciona la fragmentación externa si estamos obligados a usar asignación contigua?"

> 🗣️ **Respuesta corta:** *"Mediante la **Compactación**: el sistema operativo frena todo y corre los procesos hacia un extremo de la memoria para juntar todos los huecos libres en un solo bloque grande (como desfragmentar). El problema es que **es carísimo en tiempo de CPU** mover todo en la RAM."*

❓ 4. "¿Qué hacen la MMU y el TLB cuando Linux usa Paginación?"

> 🗣️ **Respuesta corta:** *"El programa trabaja con **direcciones lógicas (o virtuales)** pensando que su memoria es toda seguida. La **MMU (Unidad de Gestión de Memoria)** es el chip de hardware que traduce en el acto cada dirección virtual (número de página + desplazamiento) a la **dirección física** real en la RAM (número de marco + desplazamiento) mirando la Tabla de Páginas. Y como mirar la Tabla de Páginas en RAM en cada instrucción sería lento, la MMU tiene una memoria caché ultrarrápida adentro llamada **TLB** que guarda las últimas traducciones."*

❓ 5. "¿Qué es un Fallo de Página (*Page Fault*) y qué es la Hiperpaginación (*Thrashing*)?"

> 🗣️ **Respuesta corta:** *"Un **Fallo de Página** pasa cuando el programa quiere usar una página que no está cargada en la RAM (su bit de validez está en 0); eso dispara una trampa (trap) al sistema operativo para que la busque en el disco (Swap) y la suba a un marco libre. Y el **Thrashing (Hiperpaginación)** es cuando un proceso tiene tan pocos marcos de RAM asignados que se la pasa fallando y trayendo páginas del disco todo el tiempo: el disco queda al 100% y la CPU no avanza nada (como cuando abrís 50 pestañas de Chrome con 4 GB de RAM)."*